

SAP ABAP — Unit 7, Exercise 17

Modify Data Using EML

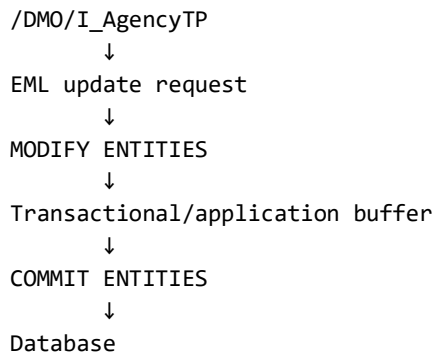
Complete practical notes: purpose, concepts, Eclipse steps, ABAP code, execution flow, commit behavior, troubleshooting, and verification.

1. Exercise Objective

Use EML (Entity Manipulation Language) to update the NAME of a travel agency through the released transactional interface /DMO/I_AgencyTP. The exercise deliberately avoids direct database-table access.

- Inspect the existing agency with Data Preview.
- Create an executable ABAP class with IF_OO_ADT_CLASSRUN.
- Prepare an EML update table.
- Use MODIFY ENTITIES to update only NAME.
- Understand why COMMIT ENTITIES is required.
- Verify the persisted value in Data Preview.

2. Transaction Flow



Key point: MODIFY ENTITIES performs the transactional modification; COMMIT ENTITIES completes the save so the change is persisted.

3. Task 1 — Analyze Current Data

1. Open /DMO/I_AgencyTP.
2. Right-click the CDS source and choose Open With → Data Preview.
3. Find AgencyID 0700##, replacing ## with your group number.
4. Record the current NAME value.

This is the before-state. Without it, you cannot reliably verify that your program changed the intended record.

4. Task 2 — Create the Global Class

5. Choose File → New → ABAP Class.
6. Select your package.
7. Name it ZCL_##_EML.

8. Description: Modify Agency Data Using EML.
9. Add interface IF_OO_ADT_CLASSRUN.
10. Finish and select the required transport request.

Why IF_OO_ADT_CLASSRUN?

It supplies IF_OO_ADT_CLASSRUN~MAIN, allowing the class to be executed directly with F9 and allowing console output through out->write().

5. Task 3 — Prepare the Update Data

DATA agencies_upd TYPE TABLE FOR UPDATE /DMO/I_AgencyTP.

This derives the correct internal-table type expected by EML for an UPDATE operation.

```
agencies_upd = VALUE #(
  ( agencyid = '070009'
    name      = 'Mustang' )
).
```

In the official exercise, replace 070009 with 0700## for your group. The new NAME can be any value.

6. MODIFY ENTITIES — Core EML

```
MODIFY ENTITIES OF /dmo/i_agencytp
  ENTITY /dmo/agency
  UPDATE FIELDS ( name )
  WITH agencies_upd.
```

Part	Meaning
MODIFY ENTITIES OF /dmo/i_agencytp	Starts an EML modification against the transactional interface.
ENTITY /dmo/agency	Uses the entity alias defined in the behavior definition.
UPDATE FIELDS (name)	Restricts the update to NAME.
WITH agencies_upd	Supplies the records and values to update.

Why /dmo/agency?

The behavior definition defines the alias /DMO/Agency. EML uses that alias in the ENTITY clause.

```
define behavior for /DMO/I_AgencyTP alias /DMO/Agency
{
  use create;
  use update;
  use delete;
}
```

7. Console Output

```
out->write( `Method execution finished!` ).
```

This only confirms that execution reached this statement. It does not by itself prove that the database was updated.

8. Why COMMIT ENTITIES Is Necessary

After the first execution, refresh Data Preview. The exercise demonstrates that MODIFY ENTITIES alone does not complete database persistence.

```
MODIFY ENTITIES
  ↓
transactional buffer
  ↓
COMMIT ENTITIES
  ↓
database persistence
```

Add:

```
COMMIT ENTITIES.
```

Then execute again and refresh Data Preview. The new NAME should now be visible in the database-backed CDS view.

9. Complete Working Method

```
METHOD if_oo_adt_classrun~main.
```

```
DATA agencies_upd TYPE TABLE FOR UPDATE /dmo/i_agencytp.
```

```
agencies_upd = VALUE #(
  ( agencyid = '070009'
    name      = 'Mustang' )
).
```

```
MODIFY ENTITIES OF /dmo/i_agencytp
  ENTITY /dmo/agency
  UPDATE FIELDS ( name )
  WITH agencies_upd.
```

```
COMMIT ENTITIES.
```

```
out->write( `Method execution finished!` ).
```

```
ENDMETHOD.
```

The AgencyID above matches the example shown in your session. Adapt it if your assigned group has a different 0700## ID.

10. Activate and Test

11. Press Ctrl + F3 to activate.
12. Press F9 to run.
13. Confirm the ABAP Console displays “Method execution finished!”.
14. Return to /DMO/I_AgencyTP Data Preview.
15. Click Refresh.
16. Find the updated AgencyID.
17. Confirm NAME now contains the new value.

11. Screenshot — Your Successful Execution

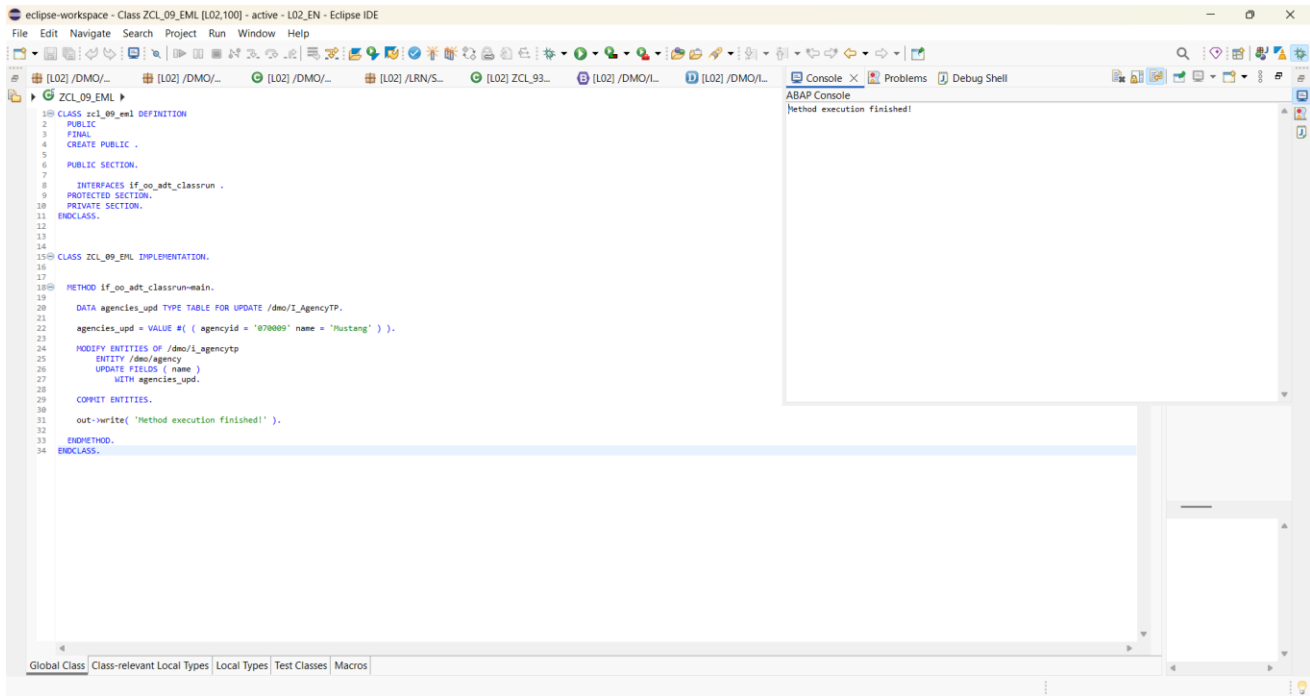


Figure 1 — Successful execution of ZCL_09_EML with console output.

12. Before vs After

Before:

AgencyID = 070009

Name = <original value>

After:

AgencyID = 070009

Name = Mustang

Only NAME is intentionally updated. Other fields are not part of UPDATE FIELDS (name).

13. Common Mistakes

Problem	Fix
Class already exists	Open the existing ZCL_##_EML class instead of creating a duplicate.
Wrong AgencyID	Use the exact 0700## ID for your group.
Wrong entity alias	Use ENTITY /dmo/agency because it is the behavior alias.
Generic update table type	Use TABLE FOR UPDATE /DMO/I_AgencyTP.
Console message appears but database is unchanged	Check that COMMIT ENTITIES is present, then refresh Data Preview.
Unexpected fields changed	Use UPDATE FIELDS (name) to restrict the modification.

14. Key Points for Practice / Interview

- EML = Entity Manipulation Language.

- RAP business objects are manipulated through EML.
- A released transactional interface can be used instead of directly accessing the underlying table.
- TABLE FOR UPDATE derives the correct EML update structure.
- VALUE #() constructs the update request.
- The behavior definition supplies the entity alias.
- UPDATE FIELDS (name) limits the fields being changed.
- MODIFY ENTITIES performs the transactional modification.
- COMMIT ENTITIES persists the transaction.
- Data Preview is used to verify the result.

15. Completion Checklist

- ☐ Checked original agency data.
- ☐ Created/opened ZCL_##_EML.
- ☐ Added IF_OO_ADT_CLASSRUN.
- ☐ Declared agencies_upd.
- ☐ Filled AgencyID and NAME.
- ☐ Used MODIFY ENTITIES.
- ☐ Used ENTITY /dmo/agency.
- ☐ Used UPDATE FIELDS (name).
- ☐ Added COMMIT ENTITIES.
- ☐ Activated and executed successfully.
- ☐ Refreshed Data Preview and verified the new NAME.